15.4.1 Restrictions on Operator Overloading

Following are some restrictions to be kept in mind while implementing operator overloading.

1. Precedence and Associativity of an operator cannot be changed.
2. No new operators can be created, only existing operators can be overloaded.
3. Cannot redefine the meaning of an operator's procedure. You cannot change how integers are added.Only additional functions can be given to an operator
4. Overloaded operators cannot have default arguments.
5. When binary operators are overloaded, the left hand object must be an object of the relevant class

Illustration 15.5 Binary operator overloading using '+'in Complex number addition

```
#include<iostream>
using namespace std;
class complex
{  int real,img;
   public:
void read()
{
cout<<"\nEnter the REAL PART : ";
cin>>real;
cout<<"\nEnter the IMAGINARY PART : ";
cin>>img;
}
complex operator +(complex c2)
{
    complex c3;
    c3.real=real+c2.real;
    c3.img=img+c2.img;
    return c3;
}
    void display()
{  cout<<real<<"+"<<img<<"i";  }
};
int main()
{
    complex c1,c2,c3;
    int choice, cont;
    cout<<"\n\nEnter the First Complex Number";
    c1.read();
    cout<<"\n\nEnter the Second Complex Number";
    c2.read();
    c3=c1+c2;  // binary + overloaded
    cout<<"\n\nSUM = ";
    c3.display();
    return 0;
 }
```

Output

```
Enter the First Complex Number
Enter the REAL PART : 3
Enter the IMAGINARY PART : 4
Enter the Second Complex Number
Enter the REAL PART : 5
Enter the IMAGINARY PART : 8
SUM = 8+12i
```


Chapter 15 Page 249-259.indd 254 3/24/2020 9:23:24 AM